

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»(ФГБОУ ВО «ВГТУ», ВГТУ)

Факультет информационных технологий и компьютерной безопасности
(факультет)

Кафедра компьютерных интеллектуальных технологий проектирования

КУРСОВОЙ ПРОЕКТ

По дисциплине «Разработка приложений для мобильных устройств»

Тема «Учет товаров в интернет магазине»

Расчетно-пояснительная записка

Выполнил студент
гр. бПО – 221

М. Р. Ивахненко
Подпись, дата

Проверил

В. В. Сокольников
Подпись, дата

Нормоконтролер

В. В. Сокольников
Подпись, дата

Защищена _____ Оценка _____
дата

Воронеж 2025

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»(ФГБОУ ВО «ВГТУ», ВГТУ)

Факультет информационных технологий и компьютерной безопасности
(факультет)

Кафедра компьютерных интеллектуальных технологий проектирования

Задание на курсовой проект

по дисциплине «Разработка приложений для мобильных устройств»

Тема работы «Учет товаров в интернет магазине»

Вариант работы 24

Студент группы бПО-221, Ивахненко Михаил Романович

Фамилия, имя, отчество

Технические условия: MS Word 2021, Android Studio 2025, PostgreSQL 17

Содержание и объем работы (графические работы, расчеты и прочее): 51 страниц, 15 рисунков, 23 таблиц, 2 приложения

Сроки выполнения этапов Рассмотрение теоретических сведений (февраль-март 2025); оформление пояснительной записки (ноябрь 2025);

Выполнил студент
гр. бПО – 221

М. Р. Ивахненко
Подпись, дата

Проверил

В. В. Сокольников
Подпись, дата

Нормоконтролер

В. В. Сокольников
Подпись, дата

Замечания руководителя

Содержание

Задание на курсовой проект	2
Замечания руководителя.....	3
Введение	6
1 Сценарии использования.....	7
1.1 Категории пользователей, их задачи	7
1.2 Общие сценарии использования приложения.....	7
1.3 Сценарии использования приложения администратором.....	9
1.4 Сценарии использования приложения обычным пользователем	11
2 Пользовательский интерфейс.....	14
2.1 Макет устройства с графом переходов.....	18
2.3 Целевые устройства, обоснование требований и подробные характеристики.....	21
3 Разработанное приложение	23
3.1 Краткое описание	23
3.2 Схема архитектуры.....	24
3.2.1 Слой UI (Уровень представления)	26
3.2.2 Доменный слой (Уровень бизнес логики)	27
3.2.3 Слой данных (Уровень источников данных)	28
3.2.4 Детальная структура компонентов	30
3.2.5 Управление зависимостями	32
3.3 Используемые технологии (внешние)	34
3.3.1 Основные технологии разработки	34
3.3.2 Сетевые библиотеки	35
3.3.3 UI и медиа библиотеки	35
3.3.4 Интеграции с внешними сервисами	36
3.3.5 Инструменты сборки и обработки кода	36

3.4	Использованные модули и системные библиотеки платформы	37
3.4.1	Архитектурные компоненты Jetpack	37
3.4.2	Kotlin библиотеки и расширения	38
3.4.3	Android Framework компоненты	39
3.4.4	Системные разрешения и API	39
4	Последовательность действий для осуществления сценариев использования	40
	Выводы	50
	Список литературы	51
	ПРИЛОЖЕНИЕ А Ссылка на репозиторий содержащий исходный код приложения	51
	ПРИЛОЖЕНИЕ Б Детализированные сценарии использования приложения	Error! Bookmark not defined.

Введение

В настоящее время цифровая экономика стремительным образом подталкивает электронную коммерцию к росту, при этом мобильные устройства становятся неотъемлемым каналом для взаимодействия интернет-магазинов с потребителями. Именно эта тенденция и обуславливает необходимость создания специализированных мобильных решений для организации эффективной торговли через платформы, покрывающие потребности малого и крафтового бизнеса.

В контексте этой тенденции возникает задача разработки комплексного программного решения, охватывающего полный цикл взаимодействия между покупателями и администраторами, которые в некоторых случаях являются владельцами интернет-магазинов. Такой инструмент должен включать, как стандартный функционал интернет-магазина, так и блок администратора для управления ассортиментом и аналитики продаж.

Предлагаемое решение представляет собой нативное андроид-приложение, построенное на современном технологическом стеке с использованием языка Kotlin. Основополагающие компоненты – декларативный UI-фреймворк Jetpack Compose и архитектурный паттерн MVVM с элементами чистой архитектуры, интеграция с Yandex Maps.

Реализация в формате мобильного приложения обусловлена возможностью использования аппаратных возможностей устройств для определения геолокации, но главный аргумент - предоставление пользователям интерфейса для взаимодействия с интернет-магазином прямо в своем смартфоне, без необходимости пользоваться браузером или desktop устройствами.

1 Сценарии использования

1.1 Категории пользователей, их задачи

Определим основные категории пользователей приложения: владелец интернет-магазина – являющийся одновременно администратором в системе (далее – администратор), и обычный пользователь – он же покупатель (далее – пользователь). Эти категории пользователей будут являться основными акторами в разрабатываемых сценариях использования. Вследствие того, что их требования к разрабатываемому программному продукту мало коррелируют между собой – разделим дальнейшие подпункты по типовым группам акторов.

Рассмотрим общий формат написания сценариев использования (от англ. Use case). У каждого такого сценария имеется общее название в формате “идентификатор: название”, использующееся в системе управления проектной документацией и непосредственно поле название, отражающее суть сценария.

Акторы – непосредственные действующие лица пользовательского сценария. Предусловия – необходимые приготовления или действия для выполнения сценария.

Основной поток – последовательность действий актора с реакцией системы на них.

Альтернативный поток – отличный от основного вариант исполнения сценария с некими другими обстоятельствами выполнения. Постусловия – состояние в которое переводятся объекты системы или акторы по результату исполнения сценария.

Исключения – реакция системы на альтернативные положительные шаги в основном или альтернативном потоке.

1.2 Общие сценарии использования приложения

Изначально рассмотрим общие сценарии использования. Для ранее рассмотренных акторов это вход и выход из учетной записи в приложении. Перейдем к сценарию, связанному со входом и регистрацией – Таблица 1.

Таблица 1 – Сценарий использования “Регистрация и авторизация”

Поле	Описание
Название	Создание учетной записи и вход в систему
Акторы	Покупатель
Предусловия	Устройство подключено к интернету
Основной поток	1. Пользователь выбирает регистрацию
	2. Вводит учетные данные
	3. Подтверждает действие
	4. Пользователь перенаправляется на страницу входа, где авторизуется с данными, указанными при регистрации
	5. Система авторизует пользователя
Альтернативные потоки	При наличии сохраненной сессии автоматический вход
	Восстановление забытого пароля
	Администратор выполняет вход, данные для авторизации устанавливаются при развертывании приложения
Постусловия	Пользователь авторизован, доступен персонализированный функционал
Исключения	Неверные учетные данные
	Учетная запись уже существует
	Отсутствие сетевого подключения

Ключевая особенность в данном сценарии заключается в подходе к регистрации в приложении – она доступна только для покупателей. Для продавца т.е. администратора изначально при развертывании создается учетная запись с известными ему параметрами, пароль и имя в последствии он сможет изменить в личном кабинете.

Таблица 2 – Сценарий использования “Смена пароля”

Поле	Описание
Название	Смена пароля
Акторы	Покупатель или Администратор
Предусловия	1. Пользователь авторизован
Основной поток	1. Пользователь переходит на страницу профиля
	2. Пользователь нажимает кнопку сменить пароль
	3. Во всплывающем диалоге пользователь вводит текущий и новый пароли
	4. Система перенаправляет пользователя на страницу входа
Альтернативный поток	3а. Пользователь отменяет смену пароля
	3а1. Диалог закрывается
	3а2. Пароль не изменяется
Постусловия	1. Все пароль пользователя изменен
	2. Сессия пользователя не прерывается
Исключения	Нет

Таблица 2 – функционал, не требующий дополнительного описания. Исключений в нем не предусматривается.

Последним общим сценарием является выход из системы – таблица 3. Он одинаков для всех типов акторов и не требует дополнительных пояснений.

Таблица 3 – Сценарий использования “Выход из системы”

Поле	Описание
Название	Выход из аккаунта пользователя
Акторы	Покупатель или Администратор
Предусловия	1. Пользователь авторизован
Основной поток	1. Пользователь переходит на страницу профиля
	2. Пользователь нажимает кнопку "Выйти"
	3. Во всплывающем диалоге пользователь подтверждает выход
	4. Система перенаправляет пользователя на страницу входа
Альтернативный поток	За. Пользователь отменяет выход:
	За1. Диалог закрывается
	За2. Пользователь остается авторизованным
Постусловия	1. Все данные авторизации удалены
	2. Корзина сохраняется локально
Исключения	Нет

1.3 Сценарии использования приложения администратором

Изначально спроектируем сценарий для управление главным функционалом приложения – менеджмент товаров и категорий. Основные особенности здесь в том, что товары здесь появляются исключения, как и в первом рассматриваемом сценарии. Они будут выражены в рамках ошибок, отображаемых на экране администратора красным текстом. Описанный сценарий – таблица 3.

Таблица 3 – Сценарий использования “Управление товарным ассортиментом”

Поле	Описание
Название	Добавление и редактирование товаров в каталоге
Акторы	Администратор
Предусловия	Администратор авторизован в системе с соответствующими правами
Основной поток	1. Администратор переходит в раздел управления товарами
	2. Выбирает действие (добавить/редактировать)

Продолжение таблицы 3.

	3. Заполняет или изменяет информацию о товаре
	4. Загружает изображения товара
	5. Сохраняет изменения
Альтернативные потоки	При редактировании загружаются существующие данные
	При удалении запрашивается подтверждение операции
Постусловия	Информация о товаре сохранена в базе данных и доступна покупателям
Исключения	Дублирование артикула товара
	Некорректный формат данных
	Ошибка загрузки изображений

Далее рассмотрим сценарий обработки заказов. Так как в рамках курсового проектирования создается минимально работоспособный продукт (он же MVP), то механизм работы с банковскими транзакциями и непосредственное управление жизненным циклом от поступления товаров на склад до доставки клиентам будет опущен – таблица 4.

Таблица 4 – Сценарий использования “Обработка заказов”

Поле	Описание
Название	Управление статусами и обработка заказов
Актеры	Администратор
Предусловия	Администратор авторизован, в системе присутствуют заказы
Основной поток	1. Администратор просматривает список заказов
	2. Выбирает конкретный заказ
	3. Изучает детали заказа
	4. Изменяет статус заказа
	5. Подтверждает изменения
Альтернативные потоки	Использование фильтров для поиска заказов
	Сортировка по различным параметрам
Постусловия	Статус заказа обновлен, покупатель уведомлен об изменении
Исключения	Попытка изменения завершенного заказа
	Ошибка обновления статуса

Сценарий “Формирование аналитики” будет покрывать модуль отчетности и желание возможного пользователя получить статистику по основным отслеживаемым за различные промежутки времени метрикам. Данный сценарий отражен в далее идущей таблице – таблица 5.

Таблица 5 – Сценарий использования “Формирование аналитики”

Поле	Описание
Название	Просмотр статистики и генерация отчетов
Акторы	Администратор
Предусловия	Администратор авторизован, в системе накоплены данные о продажах
Основной поток	1. Администратор переходит в раздел аналитики
	2. Выбирает период и тип отчета
	3. Система формирует визуализацию данных
	4. Администратор экспортирует отчет при необходимости
Альтернативные потоки	Переключение между различными представлениями данных
	Генерация PDF-отчета для печати
Постусловия	Аналитические данные представлены, отчет сохранен при экспорте
Исключения	Недостаточно данных для формирования отчета
	Ошибка экспорта

Таким образом ранее описанные сценарии покрывают основной функционал главного заинтересованного в разработке данного программного продукта лица – администратора.

1.4 Сценарии использования приложения обычным пользователем

Далее перейдем к рассмотрению сценариев использования приложения другим основным действующим пользователем приложения – покупателем. Основной функционал – оформление заказов в интернет-магазине – таблица 6.

Таблица 6 – Сценарий использования “Оформление заказа”

Поле	Описание
Название	Выбор товаров и оформление покупки
Акторы	Покупатель
Предусловия	Пользователь авторизован
Основной поток	1. Покупатель просматривает каталог товаров
	2. Добавляет выбранные товары в корзину
	3. Переходит к оформлению заказа
	4. Указывает адрес доставки
	5. Выбирает способ оплаты
	6. Подтверждает заказ
Альтернативные потоки	Использование фильтров категорий
	Выбор адреса на карте

Продолжение таблицы 6.

	Использование сохраненного адреса
Постусловия	Заказ создан и сохранен в системе, корзина очищена
Исключения	Товар отсутствует на складе
	Ошибка при оформлении заказа
	Некорректные данные доставки

После завершения заказа пользователю открывается функционал добавления к заказанным товарам отзывов. Механизм доставки, оплаты и получения опущен в рамках ранее упомянутой специфики приложения, однако сценарий, покрывающий такой функционал разработан – таблица 7.

Таблица 7 – Сценарий использования “Создание отзыва”

Поле	Описание
Название	Оценка приобретенного товара
Актеры	Покупатель
Предусловия	Заказ завершен, товар доставлен
Основной поток	1. Покупатель открывает завершенный заказ
	2. Выбирает товар для оценки
	3. Указывает рейтинг
	4. Добавляет текстовый комментарий
	5. Публикует отзыв
Альтернативные потоки	Добавление фотографий к отзыву
	Редактирование существующего отзыва
Постусловия	Отзыв опубликован и доступен другим пользователям
Исключения	Повторная попытка оценки товара
	Ошибка загрузки изображений

Последний сценарий покрывает вероятную возможность пользователя сохранить данные для будущих заказов, дабы не вводить их вручную каждый раз при заказе – таблица 8.

Таблица 8 – Сценарий использования “Управления данными для заказов”

Поле	Описание
Название	Настройка персональных данных и адресов доставки
Актеры	Покупатель
Предусловия	Пользователь авторизован
Основной поток	1. Покупатель переходит в настройки профиля
	2. Редактирует персональные данные

Продолжение таблицы 8.

	3. Добавляет или изменяет адреса доставки
	4. Сохраняет изменения
Альтернативные потоки	Установка адреса по умолчанию
	Удаление сохраненных адресов
Постусловия	Данные профиля обновлены и сохранены
Исключения	Некорректный формат данных
	Дублирование адресов

2 Пользовательский интерфейс

На этапе разработки интерфейса пользователя необходимо спроектировать макет отображаемых страниц приложения.

Для этого воспользуемся онлайн-редактором Figma, который позволяет эффективно работать с векторными и полигональными объектами для составления современных дизайн макетов с целью проектирования UI/UX.

Проектирование дизайна разрабатываемого приложения начнем с организации рабочей области – рисунок 1.

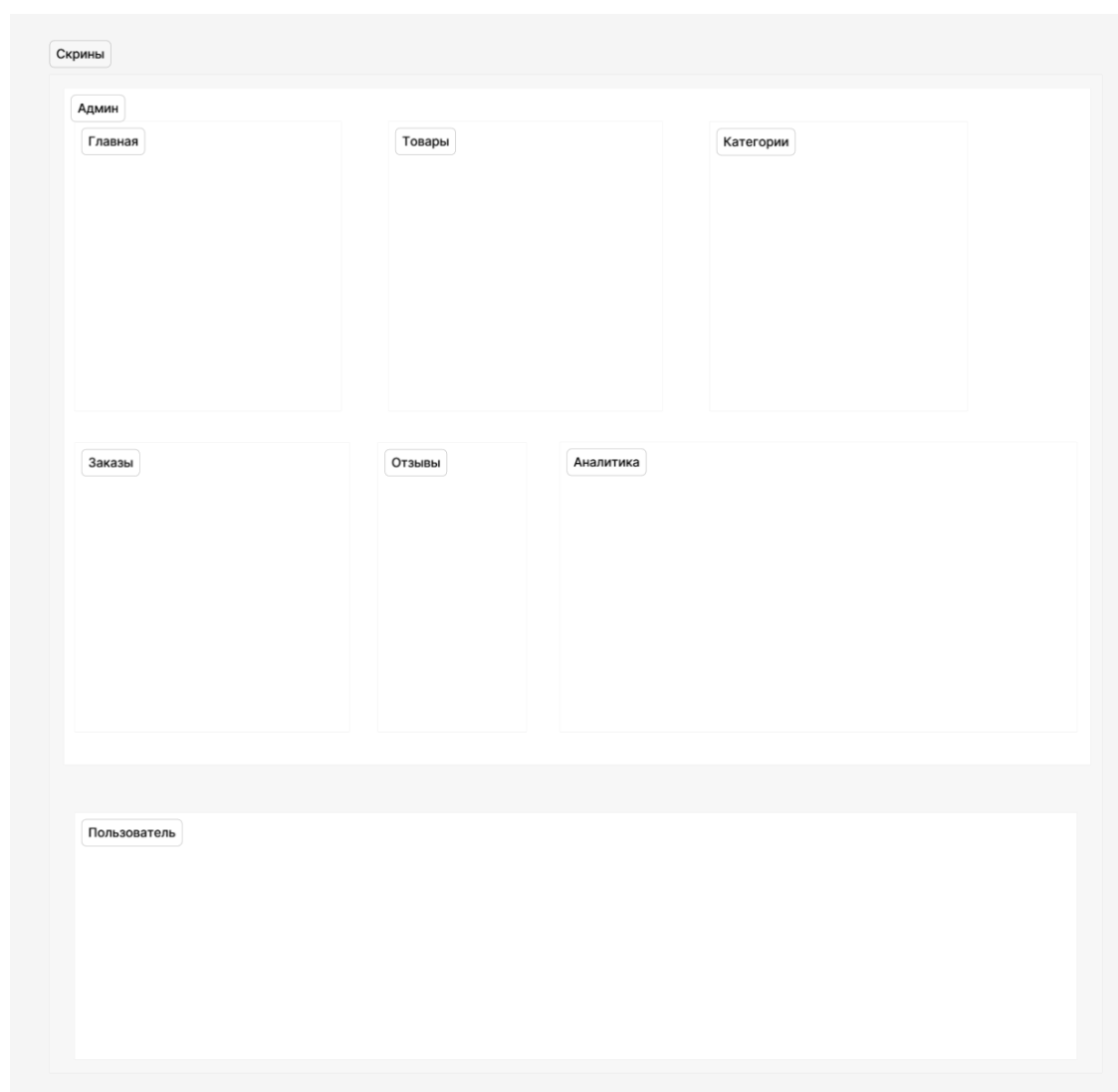


Рисунок 1 – Поделенная на секции рабочая область

Рассмотрим проектирование типового экрана приложения на примере разработки страницы входа. Изначально добавим фрейм, на котором будем компоновать наши элементы и настроим для него auto-layout, цвета и размеры. Размер выберем стандартный для многих экранов мобильных устройств – 390 на 780 пикселей. Добавленный пустой фрейм – рисунок 2.

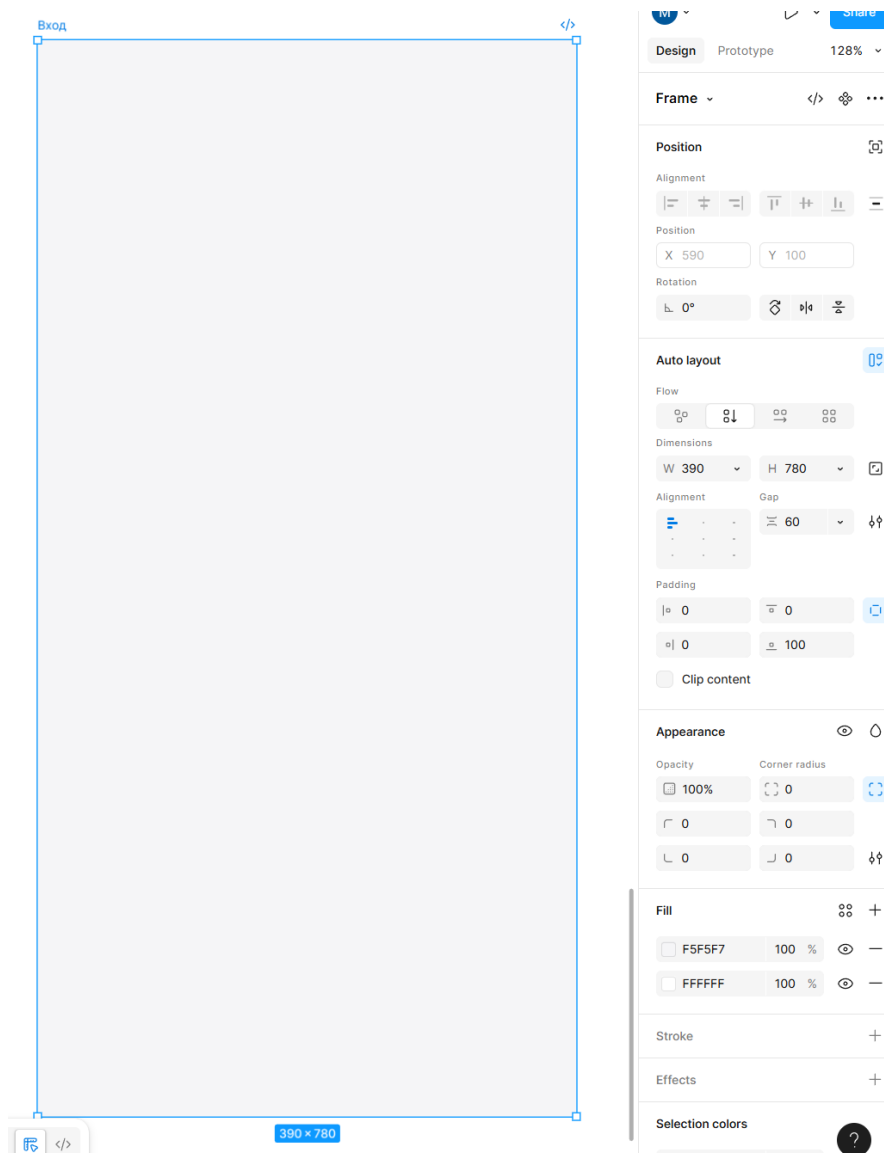


Рисунок 2 – Добавление фрейма в рамках проектирования экрана

Далее можно добавить ключевые текстовые блоки и расположить их в интуитивно понятных пользователю местах. В данном случае это будет заголовок страницы и надпись “Добро пожаловать” – рисунок 3.

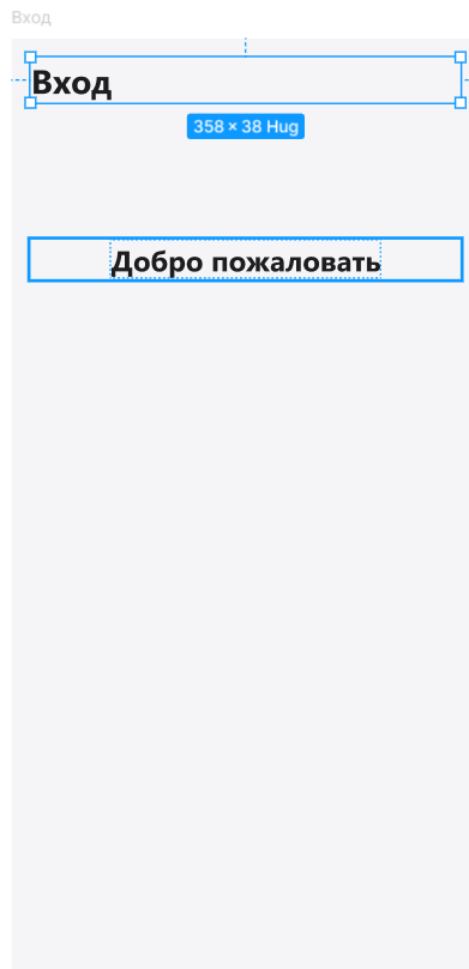


Рисунок 3 – Добавление текстовых объектов на фрейм в рамках разработки типового экрана приложения

После этого добавим все остальные элементы, а именно поля формы входа и их названия, кнопку войти и текст-ссылку на страницу регистрации. В figma каждый текстовый или шейповый блок необходимо оборачивать в блок-контейнер для последующего использования auto-layout, помогающего быстро и просто настраивать автоматическое масштабирование и расположение объектов на страницах, применив такой подход закончим разработку данного экрана – рисунок 4.

Далее в рамках разработки экрана ему необходимо добавить связи с другими экранами. Для этого разработаем экран регистрации в приложении, после чего свяжем их в режиме прототипирования и в графе переходов – рисунок 5.

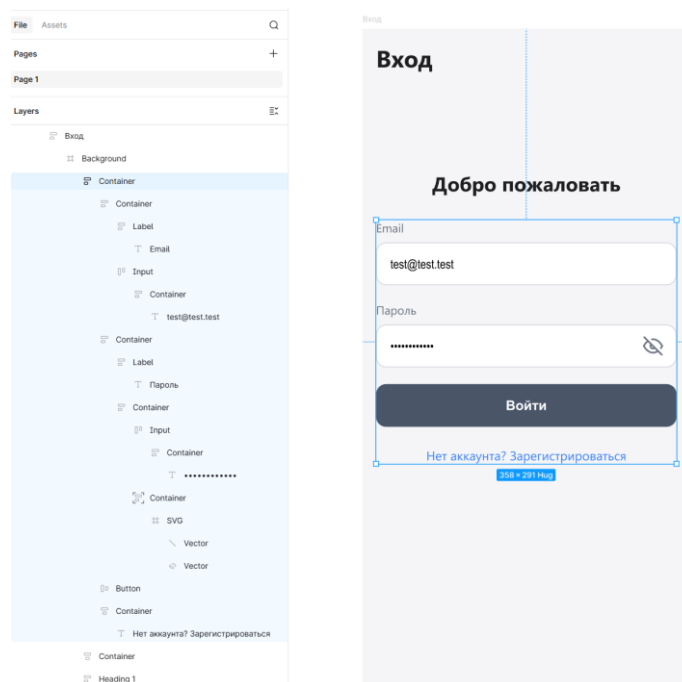


Рисунок 4 – Завершенный экран входа

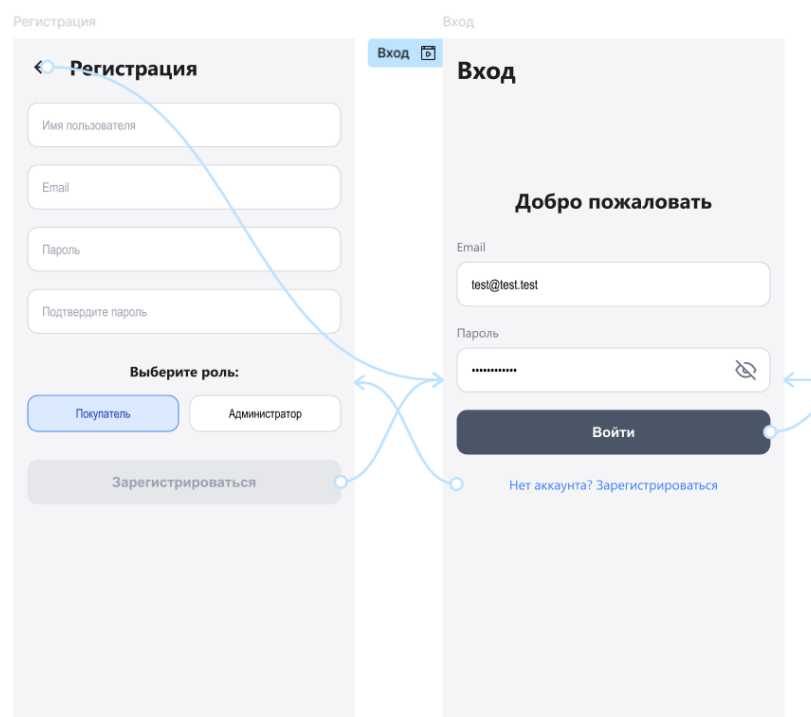


Рисунок 5 – Настроенный прототип для связи экранов входа и регистрации

Таким образом, экран входа будет являться точкой входа в систему для типового пользователя. В режиме прототипирования Figma, используя графический интерфейс можно выставить связи между экранами. В данном случае при нажатии на

кнопку “Нет аккаунта? Зарегистрироваться” добавлена связь “при нажатии переход к” и ссылка на страницу регистрации. Подобным образом выставлены связи для нажатия на стрелку назад и на кнопку “Зарегистрироваться”.

Используя подобный подход были спроектированы все остальные экраны. В них использовался подход дублирования и пере использования компонентов для последующего упрощения создания макетов непосредственно в приложении.

2.1 Макет устройства с графом переходов

Начнем рассмотрение макета с модуля регистрации/авторизации. Изначально пользователь вводит свои данные для авторизации, при их отсутствии – регистрируется. В случае успешной авторизации обычный пользователь перенаправляется на главную страницу магазина, на которой приставлены все товары магазина с возможностью сортировки по категориям. Опустим каждый раз упоминание слова “Экран” и будем использовать лишь названия экранов. Соответствующий граф, представленный на рисунке 6 можно представить в текстовом виде: Вход → Регистрация → Вход → Магазин.

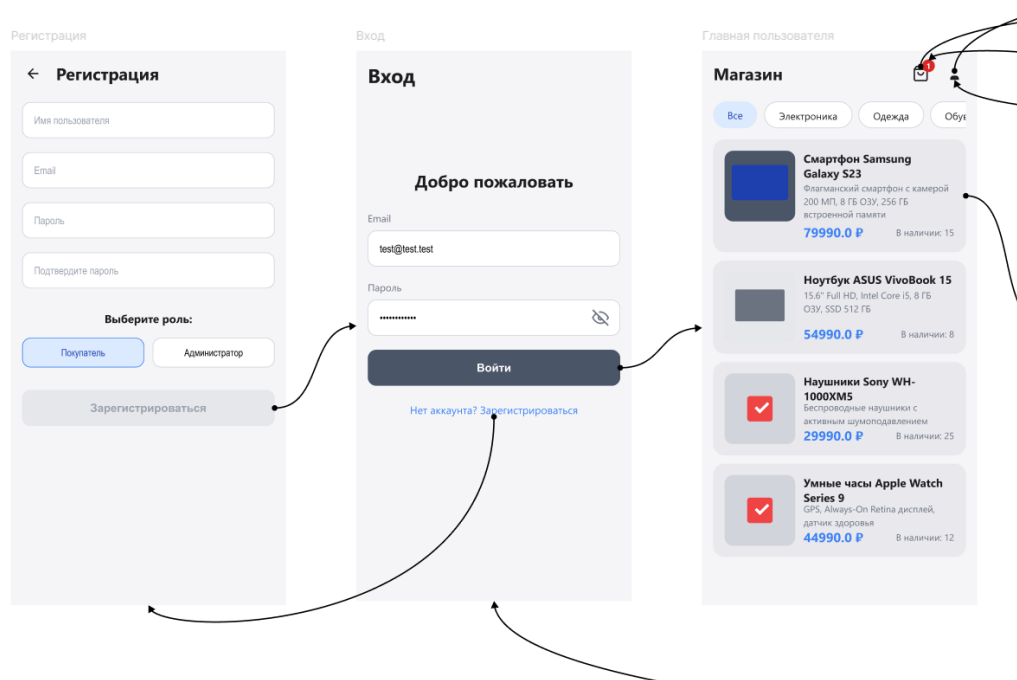


Рисунок 6 – Макет авторизации с графом перехода на главную страницу покупателя

Далее рассмотрим граф переходов “Карточка товара → Магазин → Корзина → Оформление заказа → Главная → Профиль → Мои заказы → Вход” – рисунок 7.

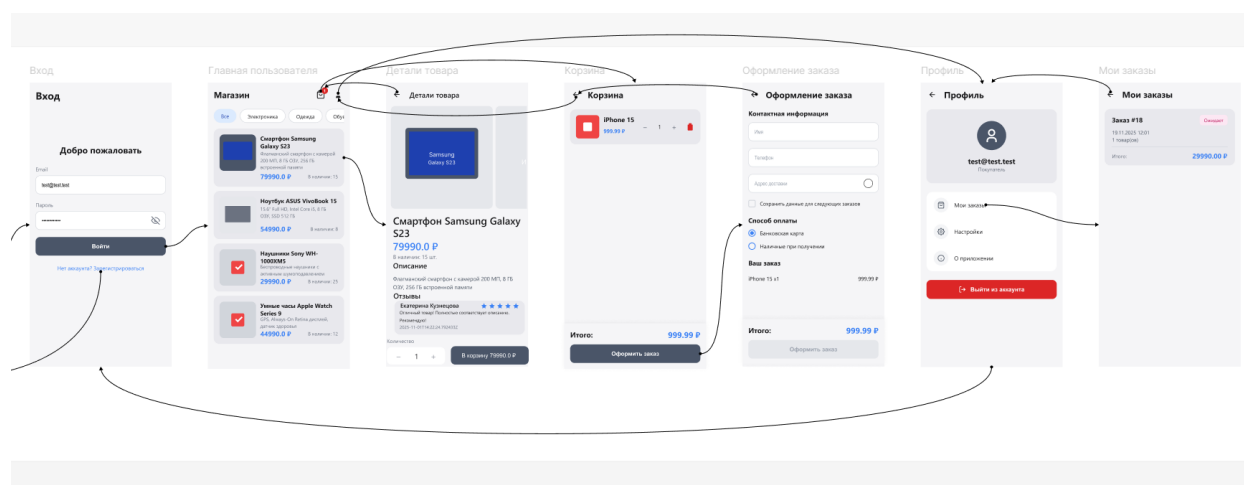


Рисунок 7 – Макет экранов покупателя по с графом переходов

Рассмотрим путь пользователя в этом графе. Первым делом пользователь выбирает необходимый товар и нажимает на его карточку, после чего добавляет его в корзину. Далее он возвращается на страницу магазина, на которой становится доступен значок корзины, при переходе в которую пользователь редактирует количество товаров, после чего нажимает на кнопку “Оформить заказ” и переходит на экран “Оформление заказа”. После ввода всех необходимых данных пользователь нажимает “Оформить заказ” и возвращается на главную, где может перейти в профиль, из которого уже и происходит переход в “Мои заказы”. В конце пути пользователя он нажимает на кнопку выйти и возвращается на главную.

Теперь рассмотрим граф переходов для администратора. Он немного проще. Основной смысл в том, что доступ к любой функциональности администратор имеет из панели управления. Таким образом граф переходов, покрывающий все потребности администратора, можно представить так: “Вход → Панель управления → Категории → Добавить категорию → Панель управления → Товары → Добавить товар → Панель управления → Заказы → Управление заказом → Панель управления → Отзывы → Панель управления → Выход”. Графически данный макет представлен на рисунке 8.



2.3 Целевые устройства, обоснование требований и подробные характеристики

В качестве целевой платформы для разработанного мобильного приложения были выбраны смартфоны. Выбор данного типа устройств обусловлен совокупностью технических, экономических и эргономических факторов, позволяющих максимально удобно пользоваться программным продуктом.

Детализируем требования и технические характеристики – таблица 9.

Таблица 9 – Требования к характеристикам целевых устройств

Параметр	Минимальное требование	Рекомендуемое значение	Обоснование
Тип устройства	Смартфон Android	Смартфон Android	Основной форм-фактор для мобильной коммерции
Версия ОС	Android 8.0 (API 26)	Android 13.0+ (API 33+)	Поддержка Jetpack Compose, охват 95%+ устройств
Target SDK	API 36	API 36	Требования Google Play, актуальные политики безопасности
Compile SDK	API 36	API 36	Доступ к новейшим API платформы
Оперативная память	2 ГБ	4+ ГБ	Стабильная работа Compose UI, кэширование данных
Постоянная память	100 МБ свободного	200+ МБ свободного	Установка приложения, локальная БД, кэш изображений
Архитектура процессора	ARM64 или x86_64	ARM64	Поддержка 64-битных библиотек
Разрешение экрана	720×1280 пикселей	1080×1920+ пикселей	Корректное отображение UI элементов
Диагональ экрана	5 дюймов	6+ дюймов	Комфортное взаимодействие с каталогом

Продолжение таблицы 9

Сенсорный экран	Touchscreen, multi-touch	Capacitive multi-touch	Жесты навигации, масштабирование изображений
Интернет-подключение	Обязательно	4G/5G или Wi-Fi	Синхронизация с REST API
GPS-модуль	Желательно	Рекомендуется	Интеграция с Yandex Maps, выбор адреса
Камера	Не требуется	Не требуется	Загрузка изображений из галереи

Для корректной работы приложения также необходим набор системных разрешений, запрашиваемых у пользователя при первом обращении к соответствующей функциональности. Разрешение INTERNET предоставляется автоматически и требуется для взаимодействия с серверным REST API [1].

Разрешения ACCESS_FINE_LOCATION и ACCESS_COARSE_LOCATION запрашиваются при использовании функции выбора адреса доставки на карте Yandex Maps. Разрешение WRITE_EXTERNAL_STORAGE необходимо только для устройств с Android 9.0 и ниже при экспорте администратором отчетов в форматах PDF, на более новых версиях Android используется современный Scoped Storage API.

Таким образом установленные системные требования обеспечивают баланс между широким охватом целевой аудитории и возможностью использования современных технологий разработки. Минимальные характеристики соответствуют параметрам бюджетных смартфонов, что делает приложение доступным для широкого круга пользователей. Рекомендуемые характеристики ориентированы на устройства среднего ценового сегмента и обеспечивают комфортную работу приложения со всеми графическими элементами, плавной анимацией переходов и быстрой загрузкой контента.

3 Разработанное приложение

3.1 Краткое описание разработанного приложения

Разработанное мобильное приложение представляет собой полнофункциональную систему, реализующую интернет-магазин для платформы Android, обеспечивающую полный цикл взаимодействия между покупателями и администраторами интернет-магазина. Приложение реализовано с использованием современного технологического стека Android-разработки на языке программирования Kotlin 2.2.20 в интегрированной среде разработки Android Studio.

Архитектура приложения построена по принципу клиент-серверного взаимодействия с четким разделением ответственности между компонентами.

Клиентская часть представляет собой нативное Android-приложение, реализованное с использованием декларативного UI-фреймворка Jetpack Compose и архитектурного паттерна MVVM (Model-View-ViewModel) в сочетании с принципами чистой архитектуры [2].

Серверная часть реализована на платформе .NET 9.0 с использованием языка C# и предоставляет REST API для взаимодействия с клиентскими приложениями.

Ключевой особенностью приложения является реализация ролевой модели доступа на уровне API, разделяющей функциональность на два основных потока: для покупателей (Customer) и администраторов (Admin). Система аутентификации построена на основе стандарта OAuth 2.0 с использованием Bearer токенов, что обеспечивает безопасный доступ к защищенным ресурсам API. При успешной аутентификации сервер выдает JWT (JSON Web Token), который клиентское приложение сохраняет в защищенном хранилище (DataStore) и включает в заголовок (Authorization) каждого последующего запроса к API [3].

Для покупателей реализован полный цикл работы с заказами, включающий просмотр каталога товаров с возможностью фильтрации по категориям, добавление товаров в корзину с локальным сохранением, оформление заказа с выбором адреса доставки на интерактивной карте Yandex Maps, отслеживание статуса заказов и

создание отзывов на приобретенные товары с возможностью прикрепления фотографий.

Для администраторов предусмотрен расширенный функционал управления: CRUD-операции с товарами и категориями, обработка заказов с изменением статусов, модерация пользовательских отзывов, просмотр аналитики продаж с визуализацией на графиках, а также экспорт данных в формате PDF для отчетности.

Хранение данных в приложении организовано по двухуровневой схеме. Основное хранилище реализовано на базе PostgreSQL на серверной стороне, что обеспечивает надежность, масштабируемость и возможность сложных транзакционных операций. Локальное хранилище на клиентской стороне реализовано с использованием библиотеки Room Persistence Library, предоставляющей объектно-реляционное отображение (ORM) для работы с SQLite базой данных. Локальная база используется для кэширования каталога товаров, сохранения содержимого корзины и ускорения загрузки часто используемых данных. Синхронизация между локальным и удаленным хранилищами осуществляется автоматически при наличии интернет-соединения, следуя стратегии offline-first, когда приложение сначала проверяет локальный кэш, а затем при необходимости обращается к серверу.

3.2 Схема архитектуры

Архитектура разработанного приложения построена на основе архитектурного паттерна MVVM (Model-View-ViewModel) в сочетании с принципами чистой архитектуры, предложенными Робертом Мартином. Данный подход обеспечивает четкое разделение ответственности между компонентами системы, независимость бизнес-логики от деталей реализации и высокую тестируемость кода [1].

Фундаментальным принципом чистой архитектуры является правило инъекции зависимостей, согласно которому зависимости в исходном коде должны указывать только внутрь, от внешних слоев к внутренним. Это означает, что внутренние слои не должны знать о существовании внешних слоев, что достигается через применение

принципа инверсии зависимостей – высокоуровневые модули зависят от абстракций, а не от конкретных реализаций.

Общая структура слоев архитектуры представлена на рисунке 9.

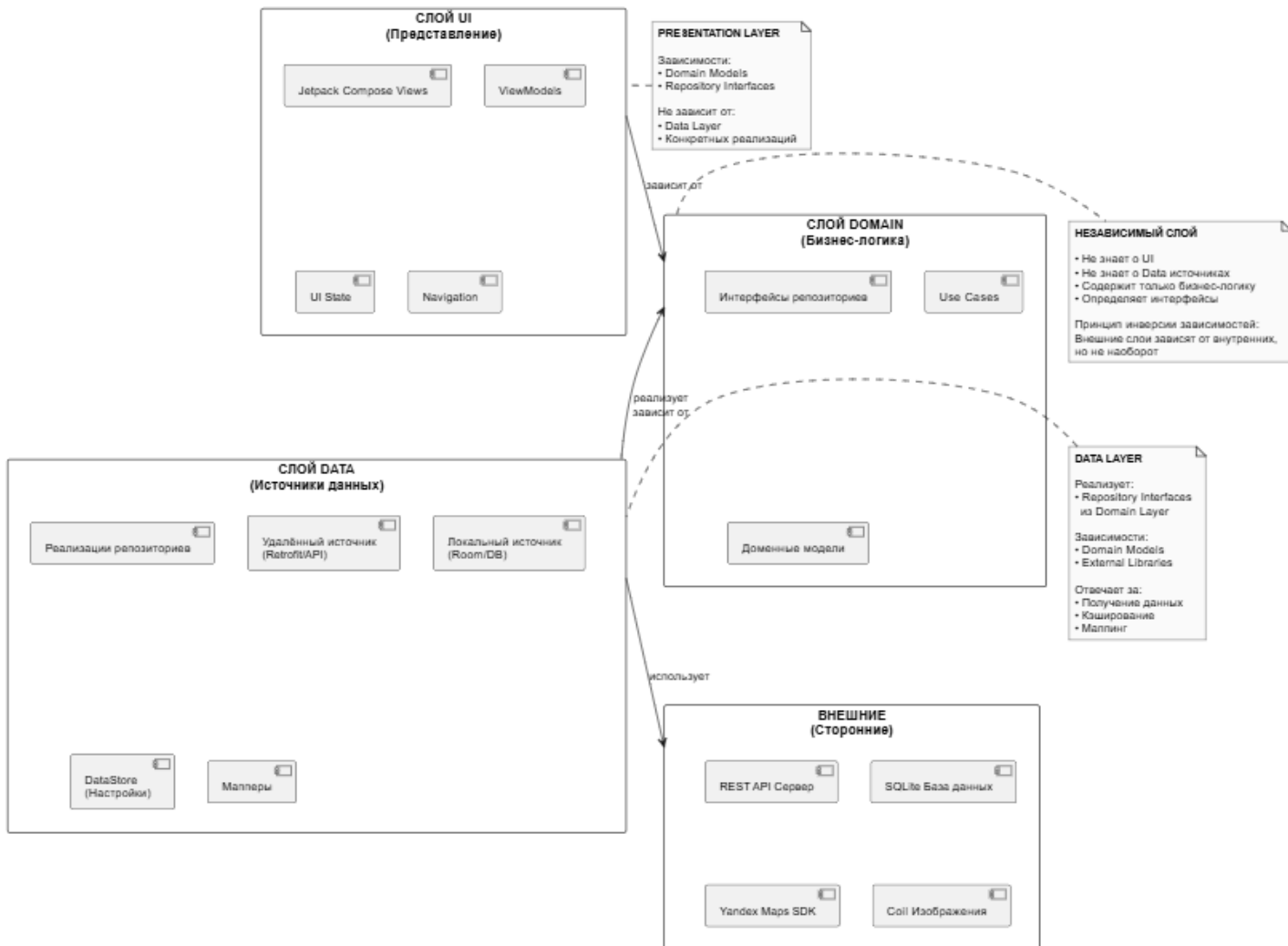


Рисунок 9 – Слоистая архитектура разработанного приложения

Как показано на рисунке 9, архитектура приложения состоит из трех основных слоев, каждый из которых имеет четко определенную ответственность и взаимодействует с другими слоями через строго регламентированные интерфейсы.

3.2.1 Слой UI (Уровень представления)

Уровень представления - верхний слой архитектуры, отвечающий за взаимодействие с пользователем и отображение информации. Этот слой реализован с использованием UI-фреймворка Jetpack Compose, который позволяет описывать пользовательский интерфейс как функции от состояния, автоматически обновляя отображение при изменении данных.

Ключевыми компонентами UI Layer являются:

- Composable экраны – представляют собой функции, помеченные аннотацией `@Composable`, которые декларативно описывают структуру и внешний вид пользовательского интерфейса. Каждый экран приложения (HomeScreen, ProductDetailScreen, CartScreen и др.) реализован как отдельная Composable функция, принимающая NavController для управления навигацией и, при необходимости, параметры маршрутизации (например, ID товара для экрана деталей) [5];

- ViewModels – компоненты, реализующие паттерн MVVM и служащие связующим звеном между UI и бизнес-логикой. ViewModels инкапсулируют логику управления состоянием экрана, обработку пользовательских действий и взаимодействие с Domain Layer через интерфейсы репозитория. Важной особенностью ViewModel является сохранение состояния при изменениях конфигурации (например, при повороте экрана), что обеспечивает бесшовный пользовательский опыт. ViewModels создаются и управляются с помощью Hilt Dependency Injection через аннотацию `@HiltViewModel` [4];

- UI State – представление состояния экрана, обычно реализованное как sealed class с несколькими вариантами: Loading (процесс загрузки данных), Success (успешная загрузка с данными), Error (ошибка с описанием). Состояние публикуется через StateFlow или LiveData, что позволяет экранам реактивно обновляться при изменении данных. Использование sealed class обеспечивает exhaustive when выражения на уровне компиляции, предотвращая забытые случаи обработки состояний;

– Navigation Compose – система управления навигацией между экранами на основе NavController и NavHost. NavHost определяет граф навигации приложения, связывая строковые маршруты (routes) с соответствующими Composable экранами. NavController предоставляет API для программного управления переходами (navigate(), popBackStack()) и автоматически управляет back stack, обеспечивая корректное поведение кнопки "Назад" [10].

Слой UI зависит исключительно от доменного слоя, взаимодействуя с бизнес-логикой через интерфейсы репозитория и используя доменные модели для представления данных. Это обеспечивает независимость презентационной логики от конкретных источников данных и позволяет легко тестировать ViewModels с использованием мок-репозитория.

3.2.2 Доменный слой (Уровень бизнес логики)

Доменный слой представляет собой центральный слой архитектуры, содержащий бизнес-логику приложения и являющийся полностью независимым от других слоев. Это самый стабильный слой архитектуры, изменения в котором происходят только при изменении бизнес-требований, но не при изменении технологий или фреймворков [7].

Ключевыми компонентами доменного слоя являются:

Repository Interfaces – интерфейсы, определяющие контракты для работы с данными. Каждый интерфейс репозитория описывает набор операций для работы с определенной доменной сущностью. Например, ProductRepository определяет методы getProducts(), getProduct(id), searchProducts(query), OrderRepository – createOrder(), getOrders(), updateOrderStatus(). Важно отметить, что это именно интерфейсы, а не конкретные реализации, что позволяет применять принцип инверсии зависимостей [8].

Domain Models – чистые бизнес-сущности, представляющие основные концепции предметной области. Это классы данных (data class в Kotlin), содержащие

только поля и, возможно, бизнес-логику валидации или вычислений. Примеры доменных моделей: Product (товар с полями id, name, price, description, imageUrl), Order (заказ с полями id, userId, items, totalPrice, status, deliveryAddress), User (пользователь), Review (отзыв). Критически важно, что доменные модели не содержат никаких зависимостей от Android framework, баз данных или сетевых библиотек – это чистые POKO (Plain Old Kotlin Objects).

Use Cases – инкапсулируют отдельные бизнес-операции или сценарии использования. Пользовательские сценарии применяются в случаях, когда бизнес-логика сложнее простого вызова метода репозитория и требует координации работы нескольких репозиториях или выполнения дополнительной обработки данных. Например, CreateOrderUseCase может включать валидацию корзины, проверку наличия товаров, вычисление итоговой стоимости с учетом скидок и создание заказа через OrderRepository.

Полная независимость доменного слоя является ключевым преимуществом архитектуры. Этот слой может быть протестирован без Android окружения, переиспользован в других проектах (например, в iOS приложении на Kotlin Multiplatform) и легко понят новыми разработчиками, так как содержит только бизнес-логику без технических деталей.

3.2.3 Слой данных (Уровень источников данных)

Слой данных представляет собой нижний слой архитектуры, отвечающий за получение данных из различных источников и их предоставление в формате, требуемом доменным слоем. Этот слой изолирует бизнес-логику от деталей работы с сетью, базами данных и другими источниками данных.

Ключевыми компонентами слоя данных являются:

Repository Implementations – конкретные реализации интерфейсов репозитория, определенных в Domain Layer. Каждая реализация репозитория координирует работу с различными источниками данных (удаленными и локальными), реализует стратегии кэширования и обработку ошибок. Например,

ProductRepositoryImpl может сначала проверить наличие актуальных данных в локальной базе (Room), и только при их отсутствии или устаревании обратиться к REST API сервера, после чего сохранить полученные данные в локальный кэш для последующего использования в offline-режиме.

Remote Data Source – компоненты для работы с удаленными данными через REST API. Включает Retrofit – type-safe HTTP клиент для Android, OkHttp – низкоуровневый HTTP клиент с поддержкой interceptors для добавления заголовков аутентификации, логирования и обработки ошибок, и API Services – интерфейсы с аннотациями Retrofit (@GET, @POST, @PUT, @DELETE), определяющие endpoints сервера. Например, ProductApi содержит метод getProducts(): List, OrderApi – метод createOrder(@Body order: OrderDto): OrderDto.

Local Data Source – компоненты для работы с локальной базой данных SQLite через библиотеку Room Persistence Library. Room предоставляет объектно-реляционное отображение (ORM), упрощающее работу с базой данных и обеспечивающее compile-time проверку SQL запросов. Включает Entity классы, представляющие таблицы базы данных (например, ProductEntity с аннотациями @Entity, @PrimaryKey), DAO (Data Access Object) интерфейсы с методами для CRUD операций (например, ProductDao с методами getAllProducts(), insertProducts(), deleteProduct()), и Database класс, расширяющий RoomDatabase и определяющий версию схемы и список Entity.

DataStore Preferences – современная замена SharedPreferences для хранения небольших объемов данных, таких как токены аутентификации, настройки приложения, флаги первого запуска [9]. DataStore обеспечивает асинхронный доступ к данным через Kotlin Coroutines и Flow, type-safety через Protocol Buffers и автоматическую миграцию при изменении схемы данных.

Data Mappers – компоненты для преобразования между различными представлениями данных на границах слоев. Mapper классы содержат функции для преобразования DTO (Data Transfer Objects, используемые для сериализации JSON) в Domain Models, Entity (используемые Room для работы с БД) в Domain Models и обратно. Например, ProductMapper содержит функции toDomain(dto: ProductDto):

Product, toEntity(domain: Product): ProductEntity, fromEntity(entity: ProductEntity): Product. Использование mappers позволяет изолировать Domain Layer от форматов сериализации и схем баз данных [3].

Data Layer реализует интерфейсы, определенные в Domain Layer, обеспечивая инверсию зависимостей – Domain зависит от абстракций, а Data предоставляет конкретные реализации этих абстракций.

3.2.4 Детальная структура компонентов

Детальная структура архитектуры с указанием конкретных технологий и взаимодействия компонентов представлена на рисунке 10.

Стрелки обозначают направление зависимостей: сплошные стрелки указывают на прямые вызовы методов, пунктирные – на реализацию интерфейсов.

Взаимодействие компонентов осуществляется следующим образом:

1. Пользователь взаимодействует с UI (Composable экранами)
2. UI передает события в ViewModel
3. ViewModel вызывает методы интерфейсов репозитория доменного уровня
4. Конкретные реализации репозитория уровня данных обрабатывают запросы
5. Репозитории координируют работу удаленных и локальных источников данных
6. Мапперы данных преобразуют данные между слоями
7. Результаты возвращаются через Flow в ViewModel
8. ViewModel обновляет UI State
9. UI автоматически перерисовывается при изменении State

Hilt Dependency Injection (показан пунктирной рамкой) автоматически создает и внедряет все зависимости, обеспечивая правильную инициализацию компонентов и управление их жизненным циклом.

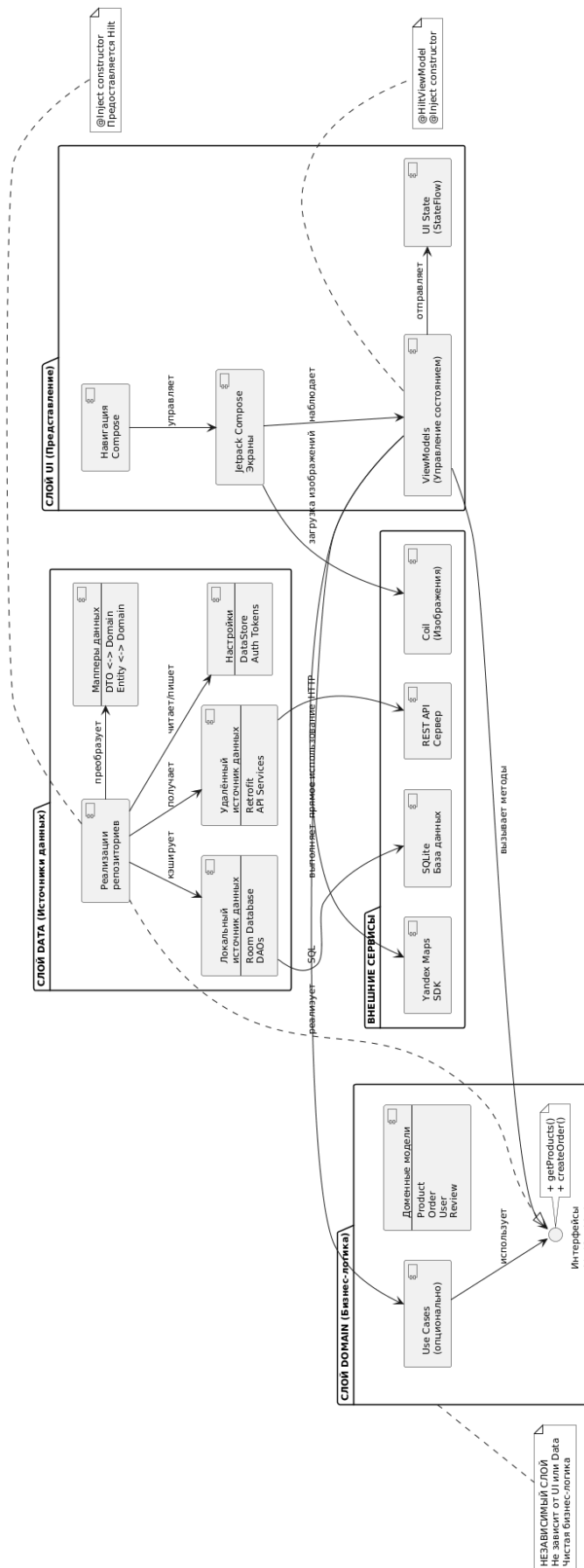


Рисунок 10 – Детальная архитектура приложения с указанием компонентов и связей

3.2.5 Управление зависимостями

Для управления зависимостями в приложении используется Hilt – официальная библиотека для инъекции зависимостей от Google, построенная на основе Dagger 2. Hilt обеспечивает автоматическое создание и внедрение зависимостей во время компиляции, что гарантирует type-safety, высокую производительность (и раннее обнаружение ошибок конфигурации).

Система управления зависимостями показана на рисунке 11.

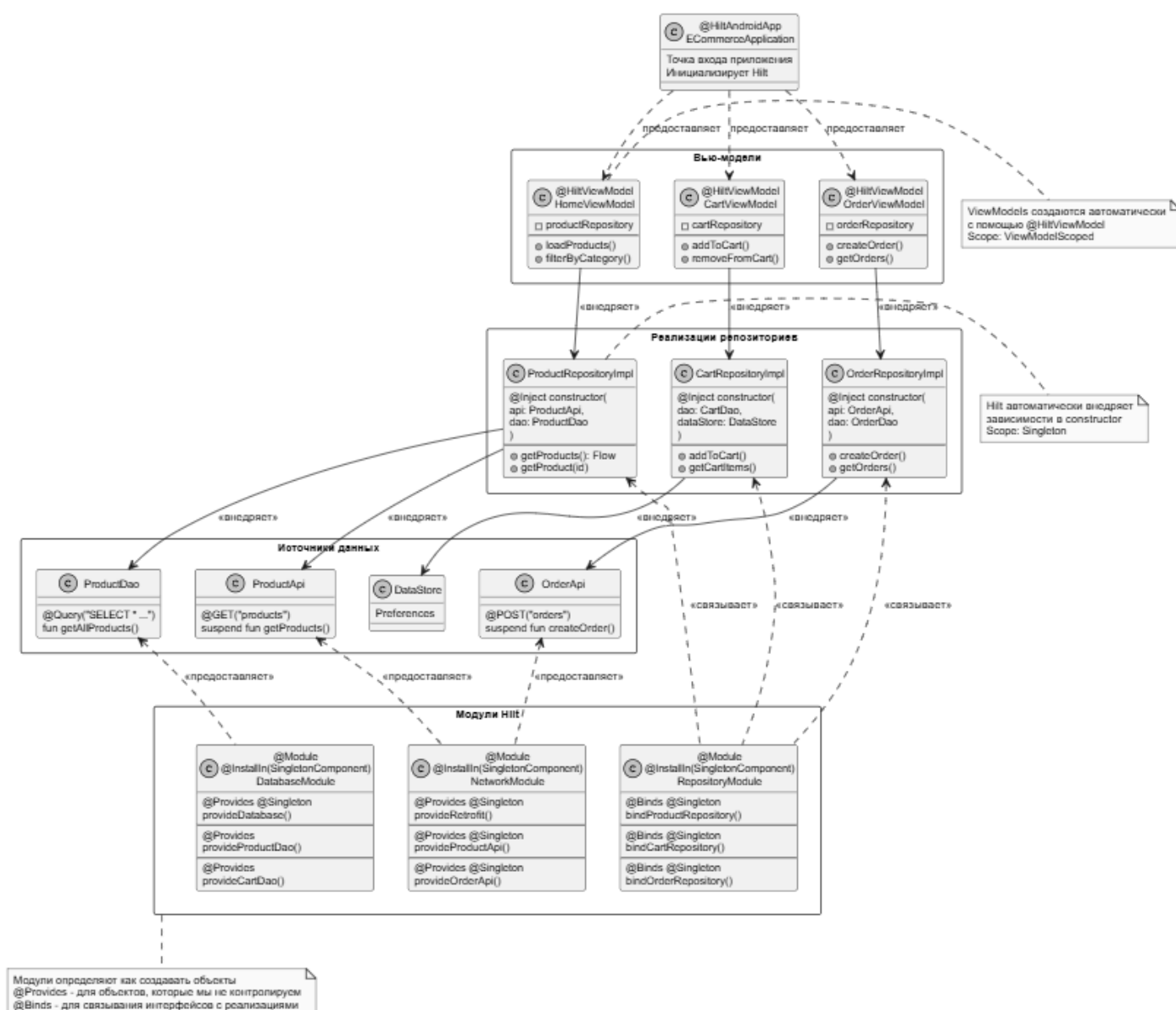


Рисунок 11 – Система Dependency Injection с использованием Hilt

Hilt работает через систему аннотаций, автоматически генерируя код для создания и внедрения зависимостей. Application класс ECommerceApplication помечается аннотацией `@HiltAndroidApp`, что инициализирует Hilt контейнер при старте приложения. ViewModels помечаются `@HiltViewModel` и получают зависимости через конструктор, помеченный `@Inject`. Репозитории и другие компоненты аналогично используют `@Inject constructor` для декларации требуемых зависимостей [6].

Для предоставления зависимостей, которые не могут быть созданы через `@Inject constructor` (например, Retrofit, Room Database, интерфейсы), используются Hilt Modules – классы с аннотацией `@Module` и `@InstallIn`, определяющие как создавать объекты.

NetworkModule предоставляет сетевые компоненты:

- `provideRetrofit()` – создает экземпляр Retrofit с базовым URL, конвертерами Gson и OkHttpClient с interceptors для добавления Bearer токена;
- `provideOkHttpClient()` – настраивает OkHttpClient с logging interceptor для отладки;
- `provideApiService()` – создает API Service из Retrofit.

DatabaseModule предоставляет компоненты базы данных:

- `provideDatabase()` – создает Room Database с миграциями и callback'ами;
- `provideCartDao()` и другие – получают DAO из базы данных.

DataStoreModule предоставляет DataStore для preferences:

- `provideDataStore()` – создает Proto DataStore для хранения токенов

Все модули помечены `@InstallIn(SingletonComponent::class)`, что означает, что предоставляемые зависимости живут на протяжении всего жизненного цикла приложения (Singleton scope). Для зависимостей с другими жизненными циклами используются `@ViewModelScoped`, `@ActivityRetainedScoped` и другие.

3.3 Используемые технологии (внешние)

В процессе разработки приложения были использованы современные технологии и библиотеки, обеспечивающие высокое качество кода, производительность и UX. Все используемые технологии являются официально рекомендуемыми Google для Android разработки или широко признанными в индустрии.

3.3.1 Основные технологии разработки

Основные технологии, используемые для разработки приложения, представлены в таблице 10.

Таблица 10 – Основные технологии разработки

Технология	Версия	Назначение
Kotlin	2.2.20	Основной язык программирования приложения
Jetpack Compose	BOM 2025.10.00	Декларативный UI фреймворк
Material Design 3	Latest	Дизайн-система UI компонентов
Android Studio	Ladybug	Интегрированная среда разработки

Kotlin 2.2.20 выбран как основной язык разработки благодаря null-safety на уровне типов, поддержке coroutines для асинхронного программирования и полной совместимости с Java библиотеками. Jetpack Compose позволяет описывать UI декларативно как функции от состояния, что упрощает разработку и поддержку пользовательского интерфейса по сравнению с XML-подходом.

3.3.2 Сетевые библиотеки

Библиотеки для работы с сетью и сериализацией данных представлены в таблице 11.

Таблица 11 – Сетевые библиотеки

Библиотека	Версия	Назначение
Retrofit	3.0.0	Type-safe HTTP клиент для REST API
OkHttp	5.2.1	HTTP клиент с interceptors
Gson	2.11.0	Сериализация/десериализация JSON
Kotlinx Serialization	1.9.0	Kotlin-native сериализация

Retrofit используется для всех HTTP запросов к серверу с автоматическим преобразованием JSON в Kotlin data classes. OkHttp interceptors применяются для добавления Bearer токена в заголовки запросов и логирования HTTP трафика в режиме отладки.

3.3.3 UI и медиа библиотеки

Библиотеки для работы с изображениями, иконками и графиками представлены в таблице 12.

Таблица 12 – UI и медиа библиотеки

Библиотека	Версия	Назначение
Coil	3.3.0	Загрузка и кэширование изображений
Material Icons Extended	Latest	Набор иконок Material Design (5000+)
Vico Charts	2.0.0-alpha.28	Построение графиков и диаграмм

Coil обеспечивает асинхронную загрузку изображений товаров с автоматическим

кэшированием. Vico Charts используется в административной панели для визуализации аналитики продаж.

3.3.4 Интеграции с внешними сервисами

Внешние сервисы и SDK представлены в таблице 13.

Таблица 13 – Интеграции с внешними сервисами

Сервис	Версия	Назначение
Yandex Maps Mobile SDK	4.8.1-lite	Отображение карт и выбор адреса доставки
Play Services Location	21.3.0	Определение геолокации пользователя

Yandex Maps Mobile SDK 4.8.1-lite – библиотека для интеграции карт Яндекс в Android приложения. Lite версия оптимизирована для мобильных устройств с меньшим размером библиотеки. Используется для интерактивного выбора адреса доставки: отображение карты в Composable, обработка нажатий на карту для выбора координат, обратное геокодирование координат в адрес, автоматическое определение текущего местоположения пользователя через GPS.

Play Services Location 21.3.0 – Google Play Services библиотека для работы с геолокацией. Предоставляет FusedLocationProviderClient для получения последнего известного местоположения и подписки на обновления локации. Используется совместно с Yandex Maps для центрирования карты на текущем местоположении пользователя.

3.3.5 Инструменты сборки и обработки кода

В проекте использовались следующие инструменты:

Gradle 8.12.3 – система сборки проекта. Управляет зависимостями, компиляцией, тестами, подписью APK. Использует Gradle Kotlin DSL (build.gradle.kts) для type-safe конфигурации.

Android Gradle Plugin 8.12.3 – плагин Gradle для Android проектов. Предоставляет задачи для компиляции Android кода.

KSP (Kotlin Symbol Processing) 2.2.20-2.0.4 – современная замена KAPT для обработки аннотаций. Используется Room, Hilt и другими библиотеками для генерации кода во время компиляции. KSP работает в 2 раза быстрее KAPT и лучше интегрирован с Kotlin компилятором.

Java 17 – минимальная версия JDK для компиляции проекта. Android Gradle Plugin 8.x требует минимум Java 17 для сборки.

Таблица использованных инструментов для сборки проекта - таблица 14.

Таблица 14 – Инструменты сборки

Инструмент	Версия	Назначение
Gradle	8.12.3	Система сборки проекта
Android Gradle Plugin	8.12.3	Плагин для Android проектов
KSP	2.2.20-2.0.4	Обработка аннотаций (замена KAPT)
Java	17	JDK для компиляции

3.4 Используемые модули и системные библиотеки платформы

Помимо внешних библиотек, в приложении используются компоненты Android Jetpack – набор библиотек от Google для упрощения Android-разработки и обеспечения обратной совместимости.

3.4.1 Архитектурные компоненты Jetpack

Hilt обеспечивает автоматическое внедрение зависимостей через аннотации (@HiltViewModel, @Inject, @Module), генерируя код DI во время компиляции. Navigation Compose управляет всеми переходами между экранами через NavHost и NavController. Room используется для локального хранения каталога товаров и

корзины с compile-time проверкой SQL запросов. DataStore применяется для хранения JWT токенов авторизации и настроек приложения. Все использованные архитектурные компоненты Android Jetpack представлены в таблице 15.

Таблица 15 – Архитектурные компоненты Jetpack

Библиотека	Версия	Назначение
Hilt (Dagger)	2.57.2	Dependency Injection для Android
Navigation Compose	2.9.6	Навигация между экранами
Lifecycle Runtime	2.9.4	Управление жизненным циклом компонентов
Room Persistence Library	2.8.2	ORM для работы с SQLite базой данных
DataStore Preferences	1.1.7	Хранение key-value данных

3.4.2 Kotlin библиотеки и расширения

Также в проекте были использованы следующие Kotlin-библиотеки:

- Kotlin Coroutines 1.9.0 – библиотека для асинхронного программирования. Позволяет писать асинхронный код в синхронном стиле с помощью suspend функций. Основные компоненты: suspend функции для асинхронных операций, CoroutineScope и coroutine builders (launch, async), Dispatchers для выбора потока выполнения (Main, IO, Default), Flow для реактивных streams данных, StateFlow/SharedFlow для hot streams. Используется повсеместно для сетевых запросов, работы с БД, обновления UI [12];

- Kotlin Collections – стандартная библиотека Kotlin для работы с коллекциями. Предоставляет функции высшего порядка (map, filter, reduce, groupBy), ленивые последовательности (Sequence), type-safe builders. Используется для обработки списков товаров, фильтрации, группировки данных.

3.4.3 Android Framework компоненты

Activity 1.9.4 – базовый компонент Android для экранов приложения. В Jetpack Compose приложения используется ComponentActivity как хост для NavHost и Compose UI. Предоставляет lifecycle callbacks, обработку system back button, интеграцию с permissions [12].

Fragment – не используется напрямую в Jetpack Compose приложениях, но Navigation Compose построен на концепциях, аналогичных Fragment navigation.

AndroidX Core-KTX 1.15.0 – библиотека Kotlin расширений для Android API. Предоставляет идиоматичные Kotlin функции для работы с Android framework, упрощенные API для SharedPreferences, Bundle, Intent, Toast и других компонентов. Пример: getString(R.string.app_name) вместо resources.getString(R.string.app_name).

3.4.4 Системные разрешения и API

Runtime Permissions – использование Android 6.0+ runtime permissions модели для запроса разрешений у пользователя. Реализовано через Accompanist Permissions библиотеку для Compose, предоставляющую rememberPermissionState() composable для декларативного управления разрешениями [11].

Location Services – использование Android Location API через FusedLocationProviderClient для получения геолокации пользователя. Требуется разрешение ACCESS_FINE_LOCATION и ACCESS_COARSE_LOCATION.

Internet Connectivity – использование ConnectivityManager для проверки доступности интернета и определения типа соединения (WiFi, Mobile Data). Позволяет адаптировать поведение приложения в зависимости от состояния сети.

Storage Access Framework (SAF) – для экспорта PDF отчетов используется SAF для предоставления пользователю выбора места сохранения файла. На Android 10+ используется Scoped Storage без запроса разрешения WRITE_EXTERNAL_STORAGE.

4 Последовательность действий для осуществления сценариев использования

4.1 Выполнение сценария: Оформление заказа (Покупатель)

4.1.1 Последовательность действий

Процесс оформления заказа от выбора товара до подтверждения покупки представлен в таблице 15.

Таблица 15 – Последовательность действий при оформлении заказа

№	Действие	Экран	Тип действия
1	Открыть приложение	-	Запуск
2	Ввести логин и пароль	LoginScreen	Ввод данных
3	Нажать кнопку "Войти"	LoginScreen	Клик
4	Просмотреть каталог товаров	HomeScreen	Просмотр
5	Выбрать категорию (опционально)	HomeScreen	Клик
6	Нажать на карточку товара	HomeScreen	Клик
7	Просмотреть детали товара	ProductDetailScreen	Просмотр
8	Выбрать количество товара	ProductDetailScreen	Ввод/клик
9	Нажать "Добавить в корзину"	ProductDetailScreen	Клик
10	Нажать иконку корзины	ProductDetailScreen	Клик
11	Проверить содержимое корзины	CartScreen	Просмотр
12	Нажать "Оформить заказ"	CartScreen	Клик
13	Ввести имя получателя	CheckoutScreen	Ввод данных
14	Ввести номер телефона	CheckoutScreen	Ввод данных
15	Нажать иконку выбора на карте	CheckoutScreen	Клик
16	Выбрать точку на карте	MapAddressPicker	Клик на карте
17	Подтвердить выбранный адрес	MapAddressPicker	Клик
18	Выбрать способ оплаты	CheckoutScreen	Клик

Продолжение таблицы 15

19	Нажать "Оформить заказ"	CheckoutScreen	Клик
20	Просмотреть подтверждение	OrdersScreen	Просмотр

Подсчитав все необходимые шаги можно выяснить, что их всего 20.

Процесс исполнения данного сценария представлен на рисунке 12.

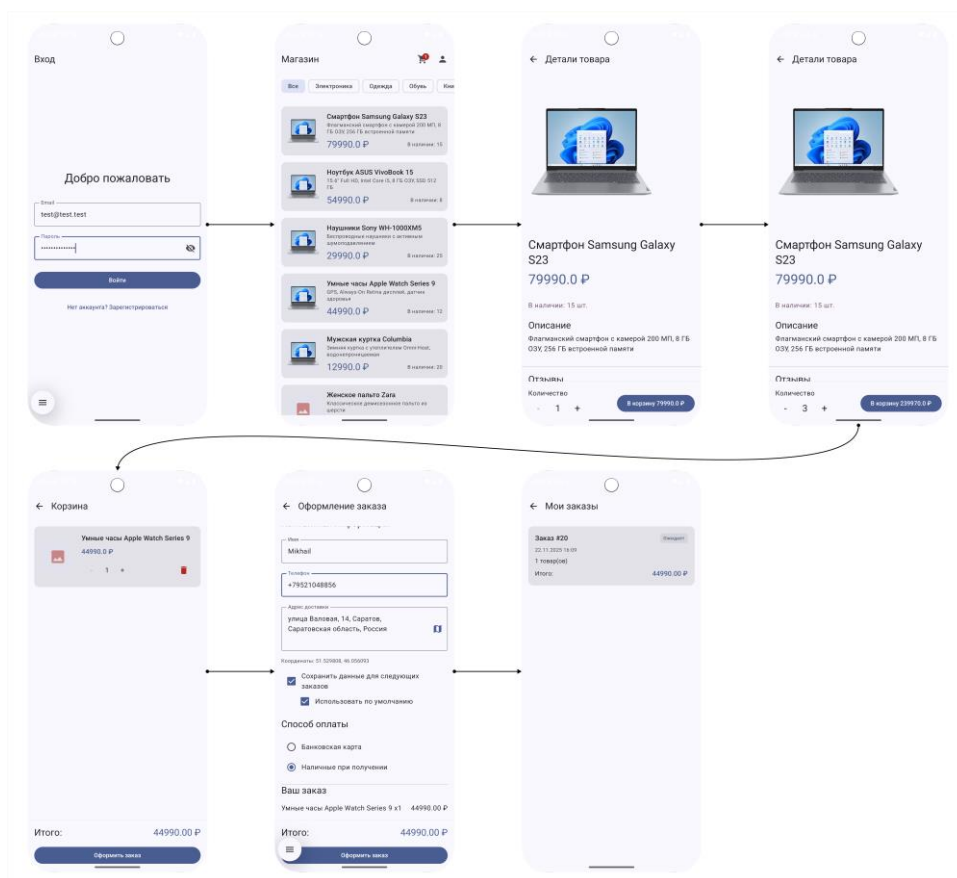


Рисунок 12 – Последовательность экранов при оформлении заказа

4.1.2 Пути сокращения последовательности действий

Для оптимизации процесса оформления заказа предлагаются следующие способы сокращения количества действий (таблица 16).

Таблица 16 – Способы оптимизации процесса оформления заказа

Метод оптимизации	Сокращение шагов	Описание реализации
Автоматический вход (сохранение сессии)	-2 шага	При сохраненной сессии пользователь сразу попадает на HomeScreen, минуя шаги 2-3
Кнопка "Купить сейчас" на карточке товара	-3 шага	Прямой переход к оформлению без промежуточного просмотра деталей и добавления в корзину (шаги 7-9)
Автозаполнение данных из профиля	-2 шага	Имя и телефон подставляются автоматически из сохраненного профиля (шаги 13-14)
Использование сохраненного адреса по умолчанию	-3 шага	Адрес выбирается из списка без необходимости работы с картой (шаги 15-17)
Запоминание способа оплаты	-1 шаг	Система использует последний выбранный способ оплаты (шаг 18)

Потенциальное сокращение: с 20 до 9 шагов (55% оптимизации)

Оптимизированный сценарий (9 шагов):

1. открыть приложение → HomeScreen (автовход);
2. нажать "Купить сейчас" на карточке товара;
3. подтвердить количество → CheckoutScreen;
4. проверить автозаполненные данные (имя, телефон, адрес);
5. проверить способ оплаты (по умолчанию);
6. нажать "Оформить заказ";
7. просмотреть подтверждение.

4.2 Выполнение сценария: Создание отзыва (Покупатель)

4.2.1 Последовательность действий

Процесс оформления отзыва представлен в таблице 17.

Таблица 17 – Последовательность действий при создании отзыва

№	Действие	Экран	Тип действия
1	Нажать иконку профиля	HomeScreen	Клик
2	Нажать "Мои заказы"	ProfileScreen	Клик
3	Найти завершённый заказ в списке	OrdersScreen	Просмотр
4	Нажать на карточку заказа	OrdersScreen	Клик
5	Нажать "Оценить товар"	OrderDetailScreen	Клик
6	Выбрать рейтинг (звезды)	ReviewDialog	Клик
7	Написать текстовый комментарий	ReviewDialog	Ввод текста
8	Добавить фотографии (опционально)	ReviewDialog	Выбор файлов
9	Нажать "Отправить отзыв"	ReviewDialog	Клик

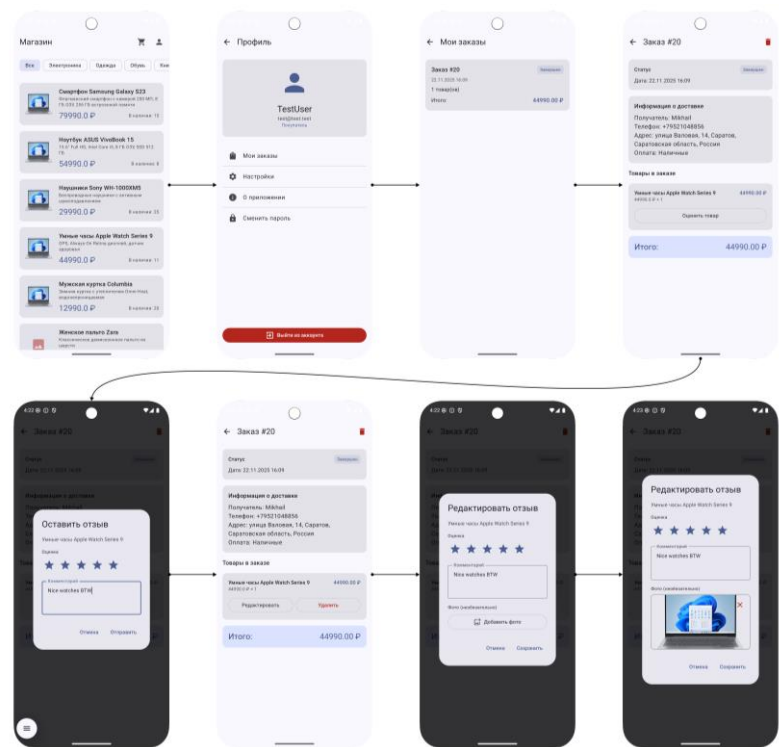


Рисунок 13 – Последовательность экранов при создании отзыва

4.2.2 Пути сокращения последовательности действий

Таблица 18 – Способы оптимизации процесса создания отзыва

Метод оптимизации	Сокращение шагов	Описание реализации
Прямая ссылка из уведомления о доставке	-4 шага	После завершения заказа пользователь получает уведомление с кнопкой "Оценить товар", переход напрямую в ReviewDialog (шаги 1-4 пропускаются)
Быстрая оценка на экране заказа	-2 шага	Звезды рейтинга доступны прямо на OrderDetailScreen без открытия диалога (шаги 5-6 объединяются)
Экспресс-отзыв (только рейтинг)	-2 шага	Возможность оставить только звездный рейтинг без обязательного текста и фото (шаги 7-8 становятся опциональными)

Потенциальное сокращение: с 9 до 3 шагов (67% оптимизации)

Оптимизированный сценарий (3 шага):

1. Нажать "Оценить" в уведомлении о доставке
2. Выбрать рейтинг звездами
3. Нажать "Отправить" (опционально добавить текст/фото)

4.3 Выполнение сценария: Добавление товара (Администратор)

4.3.1 Последовательность действий

Таблица 19 – Последовательность действий при добавлении товара

№	Действие	Экран	Тип действия
1	Войти как администратор	LoginScreen	Авторизация
2	Нажать "Товары" на Dashboard	AdminDashboard	Клик
3	Нажать кнопку "Добавить товар"	AdminProducts	Клик
4	Ввести название товара	AddProductDialog	Ввод текста
5	Ввести описание товара	AddProductDialog	Ввод текста

Продолжение таблицы 19

6	Ввести цену	AddProductDialog	Ввод числа
7	Ввести количество на складе	AddProductDialog	Ввод числа
8	Ввести артикул (SKU)	AddProductDialog	Ввод текста
9	Выбрать категорию из списка	AddProductDialog	Выбор из списка
10	Нажать "Создать товар"	AddProductDialog	Клик
11	Найти созданный товар в списке	AdminProducts	Просмотр
12	Нажать на карточку товара	AdminProducts	Клик
13	Нажать "Добавить изображение"	AdminProductEdit	Клик
14	Выбрать файлы из галереи	SystemFilePicker	Выбор файлов
15	Нажать "Сохранить изменения"	AdminProductEdit	Клик

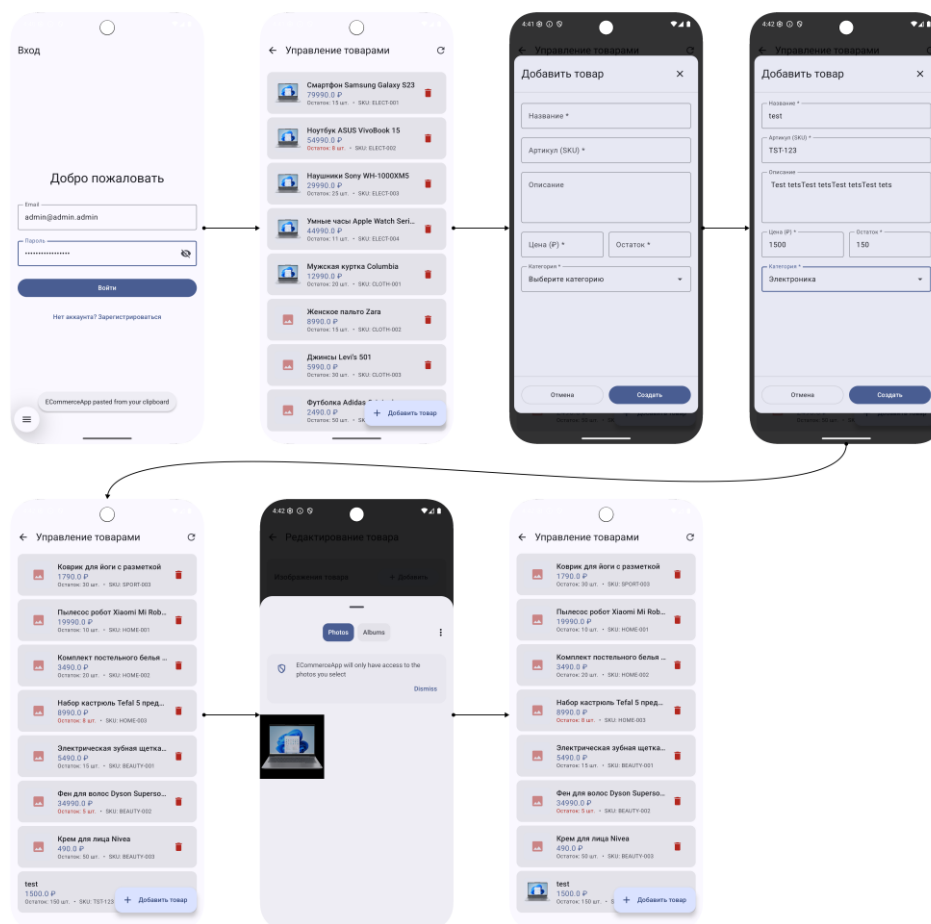


Рисунок 14 – Последовательность экранов при добавлении товара администратором

4.3.2 Пути сокращения последовательности действий

Таблица 20 – Способы оптимизации процесса добавления товара

Метод оптимизации	Сокращение шагов	Описание реализации
Сохранение сессии администратора	-1 шаг	Автоматический вход при запуске приложения (шаг 1)
Загрузка изображений в основной форме	-4 шага	Возможность добавить изображения сразу при создании товара, без необходимости повторного открытия (шаги 11-14 пропускаются)
Шаблоны товаров	-5 шагов	Создание товара на основе шаблона с предзаполненными полями категории и типичными параметрами (шаги 4-9 частично автозаполняются)
Автогенерация артикула	-1 шаг	Система автоматически генерирует уникальный SKU на основе категории и счетчика (шаг 8)
Дублирование существующего товара	-7 шагов	Копирование товара с автоматическим заполнением всех полей, кроме артикула (шаги 4-10 выполняются автоматически)

Потенциальное сокращение: с 15 до 6 шагов (60% оптимизации).

Оптимизированный сценарий (6 шагов):

1. открыть приложение → AdminDashboard (автовход);
2. товары → Дублировать похожий товар;
3. изменить название и цену;
4. изменить изображения;
5. нажать "Сохранить";
6. товар создан и опубликован.

4.4 Выполнение сценария: Обработка заказа (Администратор)

4.4.1 Последовательность действий

Таблица 21 – Последовательность действий при обработке заказа

№	Действие	Экран	Тип действия
1	Нажать "Заказы" на Dashboard	AdminDashboard	Клик
2	Найти новый заказ в списке	AdminOrders	Просмотр/поиск
3	Нажать на карточку заказа	AdminOrders	Клик
4	Просмотреть детали заказа	AdminOrderDetail	Просмотр
5	Нажать кнопку "Изменить статус"	AdminOrderDetail	Клик
6	Выбрать статус "Обрабатывается"	StatusDialog	Выбор
7	Подтвердить изменение	StatusDialog	Клик
8	Нажать "Изменить статус" повторно	AdminOrderDetail	Клик
9	Выбрать статус "Завершен"	StatusDialog	Выбор
10	Подтвердить изменение	StatusDialog	Клик

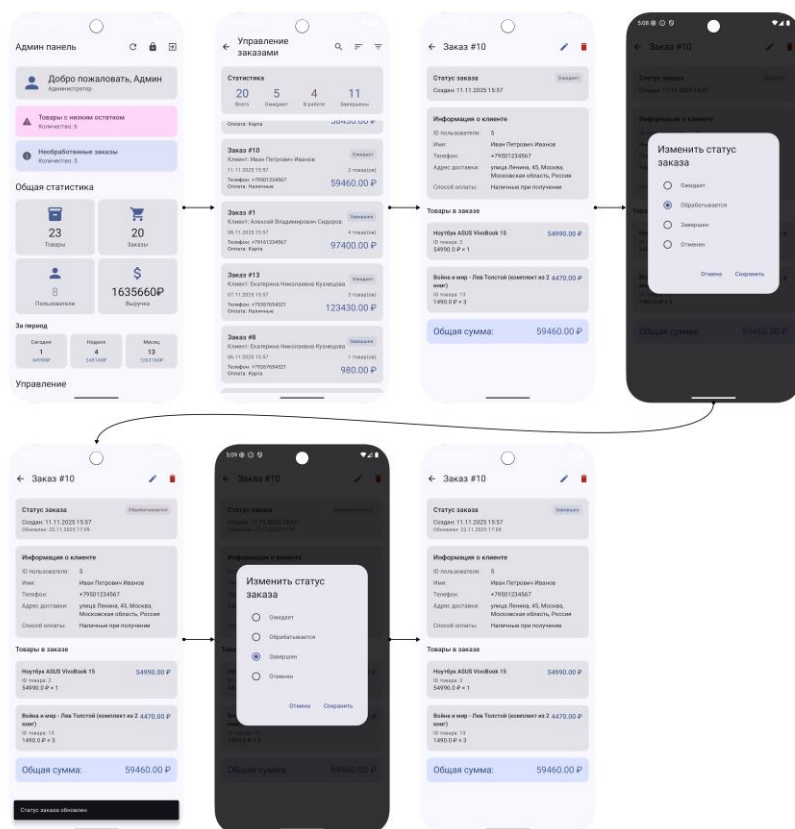


Рисунок 15 – Последовательность экранов при обработке заказа администратором

4.4.2 Пути сокращения последовательности действий

Таблица 22 – Способы оптимизации процесса обработки заказа

Метод оптимизации	Сокращение шагов	Описание реализации
Быстрые действия в списке заказов	-4 шага	Кнопки смены статуса доступны прямо на карточке заказа в списке без открытия деталей (шаги 3-7 пропускаются)
Пропуск промежуточного статуса	-3 шага	Возможность перевести заказ сразу из "Новый" в "Завершен" минуя "Обрабатывается" (шаги 5-7)
Массовые операции	-9 шагов	Возможность выбрать несколько заказов и изменить статус одновременно (для нескольких заказов)
Автоматические статусы	-10 шагов	Интеграция с системой доставки для автоматического обновления статусов (полная автоматизация)
Уведомления о новых заказах	-2 шага	Push-уведомление с быстрым действием "Принять заказ" (шаги 1-2)

Потенциальное сокращение: с 10 до 3 шагов (70% оптимизации)

Оптимизированный сценарий (3 шага):

1. Нажать "Принять" в уведомлении о новом заказе
2. Выбрать статус "Завершен" в быстром меню
3. Подтвердить изменение

4.5 Общий вывод по оптимизации приложения для повышения удобства использования

Подводя итог, можно отметить, что проведенный анализ показал возможность сокращения количества действий в среднем на 63% для основных сценариев использования. Наибольший потенциал оптимизации (70%) выявлен в административных функциях, что позволит значительно повысить эффективность управления заказами и товарами – таблица 23.

Таблица 23 – Сравнительный анализ количества шагов в сценариях

Сценарий	Текущее количество шагов	Оптимизированное количество	Сокращение	Процент оптимизации
Оформление заказа (Покупатель)	20	9	11	55%
Создание отзыва (Покупатель)	9	3	6	67%
Добавление товара (Администратор)	15	6	9	60%
Обработка заказа (Администратор)	10	3	7	70%
Среднее значение	13.5	5.25	8.25	63%

Выводы

В ходе выполнения курсовой работы была изучена архитектура MVVM для Android-приложений. Разработано мобильное приложение для учета товаров в интернет-магазине на Kotlin с использованием Jetpack Compose, позволяющее осуществлять полный цикл работы с заказами для покупателей и администраторов. Реализована система аутентификации OAuth 2.0 с Bearer токенами для взаимодействия с REST API на .NET 9.0. Применена стратегия offline-first с использованием Room для локального кэширования данных и синхронизации с PostgreSQL. Интегрированы Яндекс карты для выбора адреса доставки и сложные диаграммы для визуализации аналитики продаж. Проведена оптимизация пользовательских сценариев с сокращением среднего количества действий на 63%.

Список литературы

- 1 Голощапов, А. Л. Google Android: программирование для мобильных устройств. – СПб.: БХВ-Петербург, 2021. – 448 с.
- 2 Гриффитс, Д. Head First. Программирование для Android. – СПб.: Питер, 2022. – 704 с.
- 3 Дейтел, П. Android для программистов: создаём приложения. – СПб.: Питер, 2020. – 512 с.
- 4 Иванов, С. В. Kotlin для Android-разработчиков: Практическое руководство. – М.: ДМК Пресс, 2023. – 380 с.
- 5 Коматинени, С. Профессиональное программирование под Android. – СПб.: Символ-Плюс, 2021. – 720 с.
- 6 Меер, М. Jetpack Compose: разработка UI для Android. – М.: ДМК Пресс, 2024. – 432 с.
- 7 Петрова, Н. А. Разработка мобильных приложений на Kotlin. – М.: Бином, 2023. – 296 с.
- 8 Скин, Д. Kotlin и Android: разработка приложений. – СПб.: Питер, 2022. – 464 с.
- 9 Смирнов, А. И. Современная Android-разработка с Jetpack Compose. – Новосибирск: Сибирское университетское издательство, 2024. – 352 с.
- 10 Филлипс, Б. Android. Программирование для профессионалов. – СПб.: Питер, 2023. – 640 с.
- 11 Чейрн, В. Modern Android Development: Kotlin и архитектурные компоненты. – М.: ДМК Пресс, 2023. – 520 с.
- 12 Android Developers Documentation [Электронный ресурс]. – Режим доступа: <https://developer.android.com/docs> (дата обращения: 22.11.2025).

ПРИЛОЖЕНИЕ А Ссылки на репозиторий с исходным кодом на GitHub и GitEdu

<https://github.com/s1kko1337/cl-backend>

<https://github.com/s1kko1337/cl-mobile>

<http://git.edu.cchgeu.ru/s1kko1337/cl-backend>

[http://git.edu.cchgeu.ru /s1kko1337/cl-mobile](http://git.edu.cchgeu.ru/s1kko1337/cl-mobile)